



AST 구조 분석기 제작 보고서

📅 작성일	2026년 7월 1일
📌 과목명	프로그래밍 기초 및 응용 프로그래밍
🌀 과제	시작 전
≡ 이름	4기 39반 박성현

목차

1. 함수 개수 추출하기
2. 함수의 리턴 타입 추출하기
3. 함수의 이름 추출하기
4. 함수의 파라미터 타입 및 변수명 추출하기
5. 함수의 if 조건 개수 추출하기
6. 질문

1. 함수 개수 추출하기

구현 내용

AST의 최상위 노드인 `ext` 배열에는 함수 정의와 전역 선언이 함께 저장되어 있다.

`findFuncDef()` 함수를 구현하여 `_nodetype` 가 `FuncDef` 인 노드만 순회하도록 하였다.

찾아낸 함수의 인덱스를 저장한 뒤 최종적으로 개수를 반환하도록 구현하였다.

구현 함수

- `findFuncDef()`

```
● * 작업 실행 중: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

파일열기 성공
780194
FuncDef 개수 : 36

1회, 함수 번호 : 5
리턴 타입 : int
함수명 : main
```

2. 함수의 리턴 타입 추출하기

구현 내용

리턴 타입은 `decl → type` 내부에 존재하지만 여러 단계의 구조를 거쳐야 실제 타입에 도달할 수 있었다.

이를 해결하기 위해 `findIdentifierType()` 함수를 구현하여 `IdentifierType` 노드가 나올 때까지 재귀적으로 탐색하도록 하였다.

최종적으로 `names` 배열에서 실제 타입 값을 추출하였다.

구현 함수

- `findIdentifierType()`
- `printFunctionInfo()`

```
1회, 함수 번호 : 5
리턴 타입 : int
함수명 : main

-----파라미터 타입과 파라미터명-----
args 파라미터 없음
-----

if 노드 개수 : 0

2회, 함수 번호 : 6
리턴 타입 : char
함수명 : my_realloc
```

3. 함수의 이름 추출하기

구현 내용

함수 이름은 `decl` 노드의 `name` 필드에 저장되어 있다.

`printFunctionInfo()` 에서 해당 값을 읽어 함수명과 함께 출력하도록 하였다.

구현 함수

- `printFunctionInfo()`

```
3회, 함수 번호 : 10
리턴 타입 : void
함수명 : error

-----파라미터 타입과 파라미터명-----
args 파라미터 없음
-----

if 노드 개수 : 0

4회, 함수 번호 : 12
리턴 타입 : void
함수명 : takechar
```

4. 함수의 파라미터 타입 및 변수명 추출하기

구현 내용

파라미터 정보는 `args → params` 구조에 저장되어 있다.

`params` 배열을 순회하며 각 파라미터의 변수명과 타입을 추출하였다.

타입 역시 리턴 타입과 동일하게 `IdentifierType` 까지 재귀적으로 탐색하여 값을 얻었다.

또한 `args` 가 존재하지 않는 경우 `"args 파라미터 없음"` 을 출력하도록 예외 처리를 추가하였다.

구현 함수

- `printParameter()`
- `findIdentifierType()`

```

2회, 함수 번호 : 6
리턴 타입 : char
함수명 : my_realloc

-----파라미터 타입과 파라미터명-----
변수명: old
파라미터 타입: char

변수명: oldlen
파라미터 타입: int

변수명: newlen
파라미터 타입: int

-----

```

```

5회, 함수 번호 : 13
리턴 타입 : void
함수명 : get_token

-----파라미터 타입과 파라미터명-----
args 파라미터 없음

-----

```

⚠ 시행착오

초기에는 `args` 가 없는 경우를 고려하지 않아 런타임 오류가 발생하였다.

이를 해결하기 위해 `args.type != JSON_OBJECT` 조건을 추가하여 예외 상황을 처리하였다.

5. 함수의 if 조건 개수 추출하기

구현 내용

함수 내부 구조를 순회하기 위해 `visit()` 함수를 구현하였다.

AST는 단순한 리스트가 아닌 중첩된 트리 구조이기 때문에 재귀 탐색 방식으로 구현하였다.

탐색 방식은 다음과 같이 처리하였다.

배열 노드는 모든 원소를 탐색하고, `Compound` 는 `block_items` 를 기준으로 순회하였다.

또한 `While` 은 `stmt` 를 따라 내려가도록 구성하였다.

`If` 노드는 개수를 증가시킨 뒤 `iftrue` , `iffalse` 를 재귀적으로 탐색하도록 구현하였다.

이를 통해 모든 if 문을 누락 없이 탐색할 수 있도록 설계하였다.

🔍 AST 기반 구조 분석

if 문 개수를 검증하기 위해 실제 AST 구조를 직접 추적하며 If 노드가 어떤 경로로 탐색되는지 확인하였다.

본 분석에서는 함수 2개를 기준으로 구조를 확인하였다. 첫 번째 함수는 구조가 길어 일부까지만 분석한 뒤 중단하였고, 이후 두 번째 함수로 이어서 분석하였다.

첫 번째 함수 AST 구조 (부분 분석 후 중단)

```
- FuncDef
  - Compound
    - While
      - Compound
        - Assignment
        - While
          - BinaryOp
            - BinaryOp
              - BinaryOp
                - ID
                - Constant
              - BinaryOp
                - ID
                - Constant
            - BinaryOp
              - ID
              - Constant
          - Assignment
            - ID
          - FuncCall
            - ID
        - Assignment
        - Constant
        - While
          - BinaryOp
            - BinaryOp
              - BinaryOp
                - Constant
                - ID
              - BinaryOp
                - ID
                - Constant
            - BinaryOp
              - BinaryOp
                - Constant
                - ID
              - BinaryOp
                - ID
                - Constant
          - BinaryOp
            - ID
            - Constant
          - FuncCall
            - ID
        - If
          - BinaryOp
            - ID
            - Constant
        - If
```

두 번째 함수 AST 구조 (전체 분석)

```
- FuncDef
  - Compound
    - If
      - FuncCall
        - ExprList
          - ID
      - ID
    - Return
      - Constant
    - Compound
      - FuncCall
        - ID
      - Return
        - Constant
  - Decl
    - FuncDecl
      - ParamList
        - Decl
          - PtrDecl
          - TypeDecl
            - IdentifierType
    - TypeDecl
      - IdentifierType
```

분석 의도

위 구조를 통해 AST는 단순한 트리 구조가 아니라 `Compound`, `While`, `If` 와 같은 노드가 내부적으로 하위 구조를 가지는 중첩 구조임을 확인하였다.

특히 `If` 와 `While` 은 단순 statement가 아니라 내부에 또 다른 statement list를 포함하는 구조이기 때문에, 재귀 탐색 시 모든 branch를 포함하도록 설계해야 함을 확인하였다.

활용

이 분석 결과를 기반으로 `visit()` 함수는 DFS 방식으로 모든 하위 노드를 누락 없이 탐색하도록 구현하였다.

```

35회, 함수 번호 : 53
리턴 타입 : void
함수명 : program

-----파라미터 타입과 파라미터명-----
args 파라미터 없음
-----

if 노드 개수 : 4

36회, 함수 번호 : 54
리턴 타입 : int
함수명 : main1

-----파라미터 타입과 파라미터명-----
args 파라미터 없음
-----

if 노드 개수 : 0

```

터미널이 작업에서 다시 사용됩니다. 닫으려면 아무 키나 누르세요.

! 시행착오

초기 구현 과정에서는 AST 구조에 대한 이해 부족으로 `body` 와 같은 하위 노드를 탐색할 때 재귀 호출이 정상적으로 이루어지지 않고 자기 자신으로 다시 호출되는 문제가 발생하였다.

또한 AST에서 노드와 배열의 개념을 명확히 구분하지 못해 `stmt` 를 단순 노드로 처리하였다. 그러나 실제로 `stmt` 는 여러 하위 구조를 포함할 수 있는 요소였기 때문에 탐색 흐름이 정상적으로 이어지지 않는 문제가 있었다.

이를 해결하기 위해 AST 구조에서 노드와 배열의 차이를 구분하고, 각 구조에 맞는 재귀 탐색 방식을 적용하여 트리를 안정적으로 순회하도록 수정하였다.

6. 질문

- 본 과제에서는 if, while 등 특정 노드를 처리하기 위해 조건을 직접 분기하는 방식으로 AST를 순회하였습니다. 하지만 실제 정적 분석 도구에서는 AST 노드 종류와 패턴이 훨씬 다양하고 복잡할 것으로 생각합니다.

이처럼 조건이 매우 많아지는 경우, 모든 노드를 일일이 분기 처리하는 방식이 아니라 구조적으로 확장 가능한 방식(예: visitor pattern, rule 기반 탐색 등)을 사용하는지 궁금합니다.